claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\workflows\wf_e3cc36d8-eca\agent-abd5323fd863d305d.jsonl`
- SHA-256: `f3a4ce5555c39cb64271ab64a5d4b3756e4fd0b2024f4769cb3ec79584cf1cb6`
- Source modified: `2026-06-21T09:18:35+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_e3cc36d8-eca`
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`

Transcript

1. user (2026-06-21T09:17:04.608Z)

In the engine repo at C:/Users/avidu/Projects/badminton-highlight-indexer, map the TEST surface for locking B-P1 (gs://?/api/process?API-DB video_id) with a real regression guard like engine PR #281 (the async-job double-module hang guard).

Read: tests/test_async_jobs.py (the #281 guard ? how it drives a real job to done), tests/test_api_endpoints.py (existing /api/process tests + how they mock fetch/storage), tests/test_input_source.py, tests/test_storage.py (GCS client injection), tests/test_job_worker_module_seam.py, tests/test_job_self_scheduling.py, and run_all_tests.py / any conftest.py.

Answer precisely: (1) is there ALREADY a test asserting /api/process accepts a gcs://|gs:// URI end-to-end into the API DB? (2) which single test file is the best PATTERN to mirror for the new guard, and how does it work (TestClient vs subprocess, fixtures, monkeypatch seams, how it awaits job completion)? (3) the cleanest hermetic way to fake the GCS fetch offline (inject GCSStorage(client=fake)? monkeypatch backend.storage.fetch or get_storage to copy a local fixture? a fake backend registered via _backend_class?) ? cite the exact seam. (4) how the suite is invoked. Return structured JSON.

2. assistant (2026-06-21T09:17:06.941Z)

I'll map the test surface for locking B-P1. Let me start by reading the key test files and runner.

3. user (2026-06-21T09:17:07.907Z)

```
1       """Async job model (DEFERRED_HARDENING_PLAN #16).
2
3       Contract under test:
4       - POST /api/process ? 202 + job_id (fast-fail 4xx still happens at submit time)
5       - GET /api/jobs/{id} ? {status, progress, result, reports, ...}
6       - job `status` = EXECUTION state (queued|running|done|failed); the pipeline's own
7         verdict (e.g. failed validation) lives in result["status"].
8       - worker crash restores pre-run segments (destroy-AFTER-confirm) and marks the job
failed
9       - startup sweep (fail_stale_jobs) terminalizes jobs orphaned by a restart
10      - migration: fresh DB lands at user_version 2 with a jobs table; v1 DBs upgrade in place
11      """
12      import sqlite3
13      import time
14      from concurrent.futures import ThreadPoolExecutor
15
16      import pytest
17
18      pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
```

```
    from fastapi.testclient import TestClient

    import backend.main as m
    from backend.infrastructure.database import Database
    from backend.config.models import AppConfig
    from backend.pipeline.validators.base import ValidationResult


    class _InlineExecutor:
        """submit() runs the fn synchronously ? deterministic tests, no threads."""

        def submit(self, fn, *args, **kwargs):
            fn(*args, **kwargs)


    @pytest.fixture
    def env(tmp_path, monkeypatch):
        db = Database(str(tmp_path / "t.db"))
        cfg = AppConfig()
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        client = TestClient(m.app)
        return client, db, tmp_path, monkeypatch


    def _video(tmp_path, name="m.mp4"):
        p = tmp_path / name
        p.write_bytes(b"not-a-real-video-but-hashable")
        return p


    def _pass():
        return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]


    def _fail():
        return [ValidationResult(validator_name="blur", passed=False, message="blurry",
details={})]


    class _FakeSeg:
        def __init__(self, db, cfg):
            self.db = db

        def process_video(self, video_id, video_path, *a, **k):
            sid = self.db.add_play_segment(video_id, 0.0, 5.0)
            return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]


    class _RaisingSeg:
        def __init__(self, db, cfg):
            self.db = db

        def process_video(self, video_id, video_path, *a, **k):
            self.db.add_play_segment(video_id, 9.0, 10.0)  # half-written row pre-crash
            raise RuntimeError("boom")


    # --- migration ------------------------------------------------------------

    def test_migration_fresh_db_is_v3_with_jobs_table(tmp_path):
        db = Database(str(tmp_path / "fresh.db"))
        with db._get_connection() as conn:
```

```
            # v5 = per-user model store (PERSONALIZATION_PLAN.md §2); the jobs table arrived
at v2/v3.
            assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
            cols = {r["name"] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
        assert {"id", "video_path", "video_id", "status", "progress", "error", "result"} <=
cols
        assert {"policy", "next_poll_at"} <= cols           # v3 self-scheduling columns


    def test_migration_upgrades_v1_db_in_place(tmp_path):
        path = str(tmp_path / "v1.db")
        conn = sqlite3.connect(path)
        conn.execute(
            "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT NOT NULL, "
            "processed_at TIMESTAMP, status TEXT NOT NULL, validation_results TEXT)")
        # A genuine v1 DB also has candidate_segments (created in the version<1 block);
include it
        # so the v4 personalization ALTER (which adds user_id to that table) has a table to
alter.
        conn.execute(
            "CREATE TABLE candidate_segments (id INTEGER PRIMARY KEY AUTOINCREMENT, "
            "video_id TEXT NOT NULL, detector TEXT NOT NULL, start_time REAL NOT NULL, "
            "end_time REAL NOT NULL, duration REAL NOT NULL, stats TEXT)")
        conn.execute("PRAGMA user_version = 1")
        conn.commit()
        conn.close()

        db = Database(path)  # ladder should take it 1 ? 5
        with db._get_connection() as c:
            assert c.execute("PRAGMA user_version").fetchone()[0] == 5
            assert c.execute(
                "SELECT name FROM sqlite_master WHERE type='table' AND
name='jobs'").fetchone()
            # Restore v5 personalization coverage (user_models table + user_id on
candidate_segments).
            # These were dropped in prior edit; add (do not replace) per review.
            tables = {r[0] for r in c.execute(
                "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
            assert "user_models" in tables
            cand_cols = {r["name"] for r in c.execute(
                "PRAGMA table_info(candidate_segments)").fetchall()}
            assert "user_id" in cand_cols


    def test_parity_job_model_with_direct_processing(env, tmp_path, monkeypatch):
        """Item 5 positive: direct CLI-style path (m._execute_processing) vs job-worker path
        (submit + _InlineExecutor leading to _run_claimed_job / drain, plus explicit
_run_claimed_job)
        produce equivalent outcomes for the same ProcessRequest shape. Exercises success,
        Failure, and empty-run branches. Real DB queries using vid (no self-compares or
tautologies).
        """
        client, db, tpath, mp = env
        vid_file = _video(tpath)
        vid = m.compute_video_id(str(vid_file))
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)

        req = m.ProcessRequest(video_path=str(vid_file), segmenter="x")

        # Direct CLI-style path
        direct_out = m._execute_processing(req)
        segs = db.query_play_segments(vid, {})
        assert len(segs) >= 1, f"direct path must affect segments for vid {vid}"
        assert direct_out.get("result", {}).get("status") in (None, "success", "processed")
```

```
or "play_segments" in str(direct_out)
139
140         # Job-worker path via public API submit (exercises job creation + inline executor +
internal _run_claimed_job)
141         rj = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"x"})
142         job_out = client.get(f"/api/jobs/{rj.json()['job_id']}").json()
143         segs_j = db.query_play_segments(vid, {})
144         assert job_out["result"]["status"] == "success"
145         assert len(segs_j) >= 1
146
147         # Explicit job-worker via _run_claimed_job (create + mark running + drive the
claimed runner)
148         vid_file2 = _video(tpath, name="m2.mp4")
149         vid2 = m.compute_video_id(str(vid_file2))
150         jid = f"parity-{vid2[:8]}"
151         db.create_job(jid, str(vid_file2))
152         db.mark_job_running(jid)
153         job_row = db.get_job(jid)
154         m._run_claimed_job(job_row)
155         segs2 = db.query_play_segments(vid2, {})
156         assert len(segs2) >= 1, f"_run_claimed_job path must produce segments for vid
{vid2}"
157
158         # Failure branch (direct + job submit)
159         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
160         req_f = m.ProcessRequest(video_path=str(vid_file))
161         direct_f = m._execute_processing(req_f)
162         rf = client.post("/api/process", json={"video_path": str(vid_file)})
163         jobf = client.get(f"/api/jobs/{rf.json()['job_id']}").json()
164         assert direct_f.get("result", {}).get("status") == "failed" or
jobf["result"]["status"] == "failed"
165
166         # Empty-run branch (success verdict but zero play_segments from this execution)
167         class _EmptySeg:
168             def __init__(self, db, cfg):
169                 self.db = db
170             def process_video(self, video_id, video_path, *a, **k):
171                 return []
172         mp.setattr(m.segmenter_registry, "get", lambda name: _EmptySeg)
173         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
174         req_e = m.ProcessRequest(video_path=str(vid_file))
175         de = m._execute_processing(req_e)
176         re = client.post("/api/process", json={"video_path": str(vid_file)})
177         je = client.get(f"/api/jobs/{re.json()['job_id']}").json()
178         assert de.get("result", {}).get("status") in (None, "success")
179         assert je["result"]["status"] == "success"
180         # (reingest safety for prior on empty/fail covered in dedicated tests)
181
182
183     # --- submit-time fast-fail ------------------------------------------------------
184
185     def test_submit_returns_202_with_job_id(env):
186         client, db, tmp_path, mp = env
187         vid_file = _video(tmp_path)
188         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
189         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
190         r = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"x"})
191         assert r.status_code == 202
192         body = r.json()
193         assert body["job_id"]
194         assert body["status"] == "queued"
195         assert body["poll"].endswith(body["job_id"])
196
```

```
197
198     def test_submit_rejects_null_byte_path(env):
199         client = env[0]
200         r = client.post("/api/process", json={"video_path": "a\x00b.mp4"})
201         assert r.status_code == 400
202
203
204
205     def test_submit_missing_file_is_404(env):
206         client, db, tmp_path, mp = env
207         r = client.post("/api/process", json={"video_path": str(tmp_path / "nope.mp4")})
208         assert r.status_code == 404
209
210
211     def test_submit_unknown_segmenter_400_and_no_job_row(env):
212         client, db, tmp_path, mp = env
213         vid_file = _video(tmp_path)
214         mp.setattr(m.segmenter_registry, "get", lambda name: None)
215         mp.setattr(m.segmenter_registry, "list_available", lambda: {"motion": "..."})
216         r = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"nope"})
217         assert r.status_code == 400
218         assert "Available segmenters" in r.json()["detail"]
219         with db._get_connection() as c:
220             assert c.execute("SELECT COUNT(*) FROM jobs").fetchone()[0] == 0
221
222
223     # --- end-to-end job lifecycle (inline executor = deterministic) ---------------------
224
225     def test_job_done_with_segments_and_video_id(env):
226         client, db, tmp_path, mp = env
227         vid_file = _video(tmp_path)
228         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
229         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
230
231         job_id = client.post(
232             "/api/process", json={"video_path": str(vid_file), "segmenter":
"x"}).json()["job_id"]
233         r = client.get(f"/api/jobs/{job_id}")
234         assert r.status_code == 200
235         body = r.json()
236         assert body["status"] == "done"
237         assert body["progress"] == "finished"
238         assert body["error"] is None
239         assert body["result"]["status"] == "success"
240         assert len(body["result"]["play_segments"]) == 1
241         assert "reports" in body  # None without obsreport; key must exist per the contract
242
243         vid = m.compute_video_id(str(vid_file))
244         assert body["video_id"] == vid
245         assert len(db.query_play_segments(vid, {})) == 1
246         assert db.get_video(vid)["status"] == "processed"
247
248
249     def test_job_done_but_pipeline_verdict_failed_preserves_prior(env):
250         client, db, tmp_path, mp = env
251         vid_file = _video(tmp_path)
252         vid = m.compute_video_id(str(vid_file))
253         db.add_video(vid, str(vid_file), "processed", [])
254         db.add_play_segment(vid, 1.0, 2.0)  # prior good result
255         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
256
257         job_id = client.post("/api/process", json={"video_path":
str(vid_file)}).json()["job_id"]
258         body = client.get(f"/api/jobs/{job_id}").json()
```

```
        # Worker finished fine ? the PIPELINE verdict is the failure.
        assert body["status"] == "done"
        assert body["result"]["status"] == "failed"
        # PR-C semantics survive the async move: prior segments intact.
        assert len(db.query_play_segments(vid, {})) == 1


    def test_worker_crash_marks_failed_and_restores_prior_segments(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        vid = m.compute_video_id(str(vid_file))
        db.add_video(vid, str(vid_file), "processed", [])
        db.add_play_segment(vid, 1.0, 2.0)  # prior good result
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _RaisingSeg)

        job_id = client.post(
            "/api/process", json={"video_path": str(vid_file), "segmenter":
"x"}).json()["job_id"]
        body = client.get(f"/api/jobs/{job_id}").json()
        assert body["status"] == "failed"
        assert "RuntimeError" in body["error"]
        assert body["result"] is None
        # The half-written 9.0?10.0 row was pruned; the prior 1.0?2.0 row survives.
        segs = db.query_play_segments(vid, {})
        assert len(segs) == 1 and segs[0]["start_time"] == 1.0


    def test_progress_stages_are_recorded(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)

        stages = []
        real = db.set_job_progress

        def spy(job_id, progress):
            stages.append(progress)
            return real(job_id, progress)

        mp.setattr(db, "set_job_progress", spy)
        client.post("/api/process", json={"video_path": str(vid_file), "segmenter": "x"})
        assert "hashing" in stages
        assert any(s.startswith("segmenting") for s in stages)


    # --- polling endpoint ------------------------------------------------------

    def test_get_unknown_job_is_404(env):
        client = env[0]
        assert client.get("/api/jobs/deadbeef").status_code == 404


    # --- crash recovery --------------------------------------------------------

    def test_fail_stale_jobs_terminalizes_queued_and_running_only(env):
        client, db, tmp_path, mp = env
        db.create_job("q1", "/v.mp4")
        db.create_job("r1", "/v.mp4")
        db.mark_job_running("r1")
        db.create_job("d1", "/v.mp4")
        db.mark_job_done("d1", {"status": "success"})

        swept = db.fail_stale_jobs("interrupted by server restart")
```

```
        assert swept == 2
        assert db.get_job("q1")["status"] == "failed"
        assert db.get_job("r1")["status"] == "failed"
        assert "restart" in db.get_job("q1")["error"]
        assert db.get_job("d1")["status"] == "done"  # terminal states untouched


    # --- real executor (cross-thread SQLite/WAL sanity) ---------------------------------

    def test_real_executor_completes_job(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
        pool = ThreadPoolExecutor(max_workers=1)
        mp.setattr(m, "_get_executor", lambda: pool)
        try:
            job_id = client.post(
                "/api/process", json={"video_path": str(vid_file), "segmenter":
"x"}).json()["job_id"]
            deadline = time.time() + 10.0
            body = None
            while time.time() < deadline:
                body = client.get(f"/api/jobs/{job_id}").json()
                if body["status"] in ("done", "failed"):
                    break
                time.sleep(0.05)
            assert body is not None and body["status"] == "done"
            assert body["result"]["status"] == "success"
        finally:
            pool.shutdown(wait=True)
```

4. user (2026-06-21T09:17:08.377Z)

```
"""TestClient coverage for the FastAPI seam (previously zero ? the compiler SyntaxError
that broke `import backend.main` shipped green because nothing imported it)."""
import os
import subprocess
import sys
from unittest.mock import patch, MagicMock

import pytest

pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
from fastapi.testclient import TestClient

import backend.main as m
from backend.infrastructure.database import Database
from backend.config.models import AppConfig
from backend.pipeline.validators.base import ValidationResult


class _InlineExecutor:
    """submit() runs the fn synchronously ? async jobs complete before the response
    is inspected (full lifecycle coverage lives in test_async_jobs.py)."""

    def submit(self, fn, *args, **kwargs):
        fn(*args, **kwargs)


@pytest.fixture
def env(tmp_path, monkeypatch):
    db = Database(str(tmp_path / "t.db"))
    cfg = AppConfig()
```

```
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        client = TestClient(m.app)
        return client, db, tmp_path, monkeypatch


    def _video(tmp_path, name="m.mp4"):
        p = tmp_path / name
        p.write_bytes(b"not-a-real-video-but-hashable")
        return p


    def _pass():
        return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]


    def _fail():
        return [ValidationResult(validator_name="blur", passed=False, message="blurry",
details={})]


    class _FakeSeg:
        def __init__(self, db, cfg):
            self.db = db

        def process_video(self, video_id, video_path, *a, **k):
            sid = self.db.add_play_segment(video_id, 0.0, 5.0)
            return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]


    def test_api_config_ok(env):
        client = env[0]
        r = client.get("/api/config")
        assert r.status_code == 200
        assert r.json()["sport"] == "badminton"


    def test_process_success_with_mocked_segmenter(env):
        """#16: submit returns 202 + job_id; the (inline) job carries the old response."""
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
        r = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"x"})
        assert r.status_code == 202
        job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
        assert job["status"] == "done"
        assert job["result"]["status"] == "success"
        assert len(job["result"]["play_segments"]) == 1


    def test_process_validation_failure_preserves_prior_segments(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        vid = m.compute_video_id(str(vid_file))
        db.add_video(vid, str(vid_file), "processed", [])
        db.add_play_segment(vid, 1.0, 2.0)  # prior good result
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
        r = client.post("/api/process", json={"video_path": str(vid_file)})
        assert r.status_code == 202
        job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
        assert job["status"] == "done"  # worker finished; the PIPELINE verdict failed
```

```
93          assert job["result"]["status"] == "failed"
94          # PR-C: a failed re-validation must NOT cascade-delete prior segments.
95          assert len(db.query_play_segments(vid, {})) == 1
96
97
98      def test_process_rejects_null_byte_path(env):
99          client = env[0]
100         r = client.post("/api/process", json={"video_path": "a\x00b.mp4"})
101         assert r.status_code == 400
102
103
104     # ============================================================================
105     # Item 4 positive tests (output layout honors configured output_dir)
106     # ============================================================================
107
108     def test_process_uses_configured_output_dir_not_literal_output(env, tmp_path,
monkeypatch):
109         """Positive test for Item 4: when output.output_dir overridden, artifacts/paths
110         use the configured root; no leakage to literal 'output/' for the video_id.
111
112         This would have caught re-introduction of hard-coded os.path.join("output", ...)
113         (see main.py:548/619 and similar in compiler/detectors). Uses the existing
114         env fixture + tmp_path for isolation, matching reingest safety + compiler patterns.
115         """
116         client, db, tpath, mp = env
117         vid_file = _video(tpath)
118         vid = m.compute_video_id(str(vid_file))
119
120         custom_out = str(tmp_path / "custom_layout_out")
121         cfg = AppConfig()
122         cfg.output.output_dir = custom_out
123         mp.setattr(m, "global_config", cfg)
124
125         # ensure the custom dir is "used" (makedirs would happen in real main for CLI)
126         os.makedirs(custom_out, exist_ok=True)
127
128         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
129         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
130
131         r = client.post("/api/process", json={"video_path": str(vid_file)})
132         assert r.status_code == 202
133
134         job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
135         assert job["status"] == "done"
136         # positive: the response/result paths should reflect configured
137         assert "result" in job
138         # In real flow compiler/main would join under custom_out; here we assert config
reached
139         assert cfg.output.output_dir == custom_out
140         # use vid and test real no-leakage to literal output/<vid> (core of Item 4)
141         literal = os.path.join("output", vid)
142         assert not os.path.exists(literal), f"should not leak to literal output/ for vid
{vid}"
143         # (full FS/golden verification in owner protocol per §5/§7)
144
145
146     def test_compiler_and_reingest_safety_honor_output_dir_override(env, tmp_path,
monkeypatch):
147         """Positive for Item 4: layout check combined with reingest safety pattern.
148         Override output_dir; failed run must not pollute literal 'output/'; success uses
configured.
149         Mirrors plan to use test_api_endpoints + reingest safety + compiler.
150         """
151         client, db, tpath, mp = env
152         vid_file = _video(tpath)
```

```
        vid = m.compute_video_id(str(vid_file))

        custom_out = str(tmp_path / "reingest_custom_out")
        cfg = AppConfig()
        cfg.output.output_dir = custom_out
        mp.setattr(m, "global_config", cfg)
        os.makedirs(custom_out, exist_ok=True)

        # first "run" that fails validation (reingest safety)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
        r = client.post("/api/process", json={"video_path": str(vid_file)})
        assert r.status_code == 202
        job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
        assert job["result"]["status"] == "failed"
        # prior good would be preserved (existing test); here assert config dir was the
context
        assert cfg.output.output_dir == custom_out
        # use vid and test no leakage in the fail path too
        literal = os.path.join("output", vid)
        assert not os.path.exists(literal), f"should not leak to literal output/ for vid
{vid}"

        # success path also uses configured (no literal output/ introduced)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        r2 = client.post("/api/process", json={"video_path": str(vid_file)})
        assert r2.status_code == 202
        assert cfg.output.output_dir == custom_out  # still honored
        # use vid and test no leakage in success path
        literal = os.path.join("output", vid)
        assert not os.path.exists(literal), f"should not leak to literal output/ for vid
{vid}"



    def test_query_returns_segments(env):
        client, db, tmp_path, mp = env
        db.add_video("vid", "/x.mp4", "processed", [])
        db.add_play_segment("vid", 1.0, 4.0)
        r = client.post("/api/query", json={"video_id": "vid"})
        assert r.status_code == 200
        assert len(r.json()["play_segments"]) == 1


    def test_compile_rejects_traversal_output_filename(env):
        client, db, tmp_path, mp = env
        db.add_video("vid", str(_video(tmp_path)), "processed", [])
        sid = db.add_play_segment("vid", 1.0, 4.0)
        r = client.post("/api/compile", json={
            "video_id": "vid", "segment_ids": [sid], "output_filename": "../evil.mp4"})
        assert r.status_code == 400


    def test_compile_success_with_mocked_compiler(env):
        client, db, tmp_path, mp = env
        db.add_video("vid", str(_video(tmp_path)), "processed", [])
        sid = db.add_play_segment("vid", 1.0, 4.0)

        class _FakeCompiler:
            def __init__(self, out_dir, watermark=None, **kwargs):
                pass

            def compile_highlights(self, raw, pairs, name):
                return f"output/{name}"

        mp.setattr(m, "VideoCompiler", _FakeCompiler)
```

```
        r = client.post("/api/compile", json={
            "video_id": "vid", "segment_ids": [sid], "output_filename": "out.mp4"})
        assert r.status_code == 200
        body = r.json()
        assert body["filepath"] == "output/out.mp4"
        # the browser-reachable URL is returned so the SPA needn't hand-build it (P6)
        assert body["download_url"] == "/api/highlights/vid/out.mp4"


    def test_health_ok(env):
        client = env[0]
        r = client.get("/api/health")
        assert r.status_code == 200
        body = r.json()
        assert body["status"] == "ok"
        assert body["sport"] == "badminton"
        assert "default_segmenter" in body  # readiness fields present for the SPA banner


    def test_compile_enforces_config_min_shot_count(env):
        """stitching.min_shot_count applies on the API path (parity with
        run_process_and_stitch.py): below-floor segments are dropped; segments
        WITHOUT shot_count metadata are exempt (explicitly requested ids must not
        silently vanish under the default floor)."""
        client, db, tmp_path, mp = env
        m.global_config.stitching.min_shot_count = 8
        db.add_video("vid", str(_video(tmp_path)), "processed", [])
        s_low = db.add_play_segment("vid", 1.0, 4.0, None, None, {"shot_count": 5})
        s_high = db.add_play_segment("vid", 10.0, 14.0, None, None, {"shot_count": 12})
        s_unknown = db.add_play_segment("vid", 20.0, 24.0)  # no shot_count metadata

        pairs_seen = []

        class _FakeCompiler:
            def __init__(self, out_dir, **kwargs):
                pass

            def compile_highlights(self, raw, pairs, name):
                pairs_seen.extend(pairs)
                return f"output/{name}"

        mp.setattr(m, "VideoCompiler", _FakeCompiler)
        r = client.post("/api/compile", json={
            "video_id": "vid", "segment_ids": [s_low, s_high, s_unknown],
            "output_filename": "reel.mp4"})
        assert r.status_code == 200
        assert (10.0, 14.0) in pairs_seen and (20.0, 24.0) in pairs_seen
        assert (1.0, 4.0) not in pairs_seen


    def test_compile_400_when_all_below_min_shot_count(env):
        client, db, tmp_path, mp = env
        m.global_config.stitching.min_shot_count = 8
        db.add_video("vid", str(_video(tmp_path)), "processed", [])
        sid = db.add_play_segment("vid", 1.0, 4.0, None, None, {"shot_count": 3})
        r = client.post("/api/compile", json={
            "video_id": "vid", "segment_ids": [sid], "output_filename": "reel.mp4"})
        assert r.status_code == 400
        assert "min_shot_count" in r.json()["detail"]


    def test_compile_uses_configured_stitching_paddings(env):
        """config.json stitching.{begin,end}_padding must reach the ffmpeg trim boundaries.
        Regression for the 2026-06-10 audit finding (d): /api/compile constructed
        VideoCompiler without the paddings, so reels always used the constructor
```

```
        defaults (0.5/1.0) and the config values were silently ignored."""
        client, db, tmp_path, mp = env
        cfg = m.global_config
        cfg.output.output_dir = str(tmp_path / "out")
        cfg.stitching.begin_padding = 2.0
        cfg.stitching.end_padding = 3.0
        cfg.stitching.watermark.enabled = False  # keep the trim command minimal
        db.add_video("vid", str(_video(tmp_path)), "processed", [])
        sid = db.add_play_segment("vid", 10.0, 14.0)

        calls = []

        def fake_run(cmd, *a, **k):
            calls.append(cmd)
            if cmd[0] == "ffprobe":
                return MagicMock(stdout="100.0\n", returncode=0)
            out = cmd[-1]
            if isinstance(out, str) and out.endswith(".mp4"):
                open(out, "wb").write(b"clip")
            return MagicMock(returncode=0, stdout=b"", stderr=b"")

        with patch.object(subprocess, "run", side_effect=fake_run):
            r = client.post("/api/compile", json={
                "video_id": "vid", "segment_ids": [sid], "output_filename": "reel.mp4"})
        assert r.status_code == 200

        trim = next(c for c in calls if c[0] == "ffmpeg" and "-ss" in c)
        # 10.0 - begin_padding 2.0 and 14.0 + end_padding 3.0; the unwired constructor
        # defaults (0.5/1.0) would have produced 9.5/15.0 here.
        assert trim[trim.index("-ss") + 1] == "8.0"
        assert trim[trim.index("-to") + 1] == "17.0"


    def test_parity_job_worker_vs_direct_cli_path(env, tmp_path, monkeypatch):
        """Item 5 positive: same ProcessRequest through job worker path and direct CLI-style
path
        produce byte-identical DB rows, reports, final play_segments (modulo timestamps).
        Exercises Failure and empty-run branches.
        """
        client, db, tpath, mp = env
        vid_file = _video(tpath)
        vid = m.compute_video_id(str(vid_file))
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
        # job worker path
        rj = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"x"})
        jobj = client.get(f"/api/jobs/{rj.json()['job_id']}").json()
        # direct CLI-style path (using same processing setup as main CLI; inline executor
makes outcomes identical in test harness)
        rd = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"x"})
        jobd = client.get(f"/api/jobs/{rd.json()['job_id']}").json()
        # byte-identical key outcomes
        assert jobj["result"]["status"] == jobd["result"]["status"]
        assert len(db.query_play_segments(vid, {})) == len(db.query_play_segments(vid, {}))
        # for failure
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
        rf = client.post("/api/process", json={"video_path": str(vid_file)})
        jobf = client.get(f"/api/jobs/{rf.json()['job_id']}").json()
        assert jobf["result"]["status"] == "failed"
        # empty-run similar (would preserve prior via reingest safety in both paths)


    # ------------------------------------------------------------------ M2: URI-aware input
```

```
    class _CaptureSeg:
        """Records the video_path the segmenter actually received (M2: must be a LOCAL
path)."""
        last_path = None

        def __init__(self, db, cfg):
            self.db = db

        def process_video(self, video_id, video_path, *a, **k):
            _CaptureSeg.last_path = video_path
            sid = self.db.add_play_segment(video_id, 0.0, 5.0)
            return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]


    def test_process_local_input_passes_identity_path(env):
        """Default (local) input must reach the segmenter as the EXACT same path ? identity,
no copy
        (the M2 zero-behaviour-change guarantee for today's local workflow)."""
        client, db, tmp_path, mp = env
        vid = _video(tmp_path)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
        _CaptureSeg.last_path = None
        r = client.post("/api/process", json={"video_path": str(vid), "segmenter": "x"})
        assert r.status_code == 202
        assert _CaptureSeg.last_path == str(vid)


    def test_process_local_missing_file_still_404(env):
        """A LOCAL input that doesn't exist still fast-fails 404 ? the existence gate is
preserved
        for local paths (only non-local URIs skip it)."""
        client = env[0]
        r = client.post("/api/process", json={"video_path": "/no/such/local/file.mp4"})
        assert r.status_code == 404


    def test_process_nonlocal_uri_skips_exists_and_fetches(env):
        """A gs:// input is NOT 404'd at submit (can't os.path.exists a bucket object); the
worker
        fetches it to a local scratch file and the segmenter sees that LOCAL path, not the
URI."""
        client, db, tmp_path, mp = env

        def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
            with open(dst, "wb") as fh:
                fh.write(b"fetched-video-bytes")
            return dst

        mp.setattr(m.storage, "fetch", fake_fetch)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
        _CaptureSeg.last_path = None
        r = client.post("/api/process", json={"video_path": "gs://bucket/videos/x.mp4",
"segmenter": "x"})
        assert r.status_code == 202                              # NOT 404 ? non-local skips
the exists gate
        job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
        assert job["status"] == "done"
        assert _CaptureSeg.last_path is not None
        assert not _CaptureSeg.last_path.startswith("gs://")     # a fetched LOCAL path, not
the URI
        assert os.path.basename(_CaptureSeg.last_path) == "x.mp4"
        # Provenance: the DB record keeps the SOURCE URI (not the ephemeral scratch path).
```

```
397         rec = db.get_video(job["video_id"])
398         assert rec is not None and rec["filepath"] == "gs://bucket/videos/x.mp4"
399
400
401     def test_process_unknown_scheme_returns_400(env):
402         """An unsupported scheme (https/ftp/file) is a clean 400, never a 500 ? parse_uri's
ValueError
403         must not escape the request handler."""
404         client = env[0]
405         for bad in ("https://h/x.mp4", "ftp://h/x.mp4", "file:///etc/passwd"):
406             r = client.post("/api/process", json={"video_path": bad})
407             assert r.status_code == 400, f"{bad} -> {r.status_code}"
408
409
410     def test_process_local_scheme_uri_accepted(env):
411         """An explicit local://<existing file> resolves to the bare path: the existence gate
stats the
412         RESOLVED key (not the raw URI) so it is accepted, and the segmenter sees the
stripped path."""
413         client, db, tmp_path, mp = env
414         vid = _video(tmp_path, "ls.mp4")
415         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
416         mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
417         _CaptureSeg.last_path = None
418         # Forward-slash local:// URI (parse strips the scheme to the bare path).
419         uri = "local://" + str(vid).replace("\\", "/")
420         r = client.post("/api/process", json={"video_path": uri, "segmenter": "x"})
421         assert r.status_code == 202
422         assert _CaptureSeg.last_path == str(vid).replace("\\", "/")
423
424
425     def test_process_local_scheme_uri_missing_still_404(env):
426         """local://<missing> still 404s ? the existence gate runs on the resolved local
key."""
427         client = env[0]
428         r = client.post("/api/process", json={"video_path": "local:///no/such/ls-
missing.mp4"})
429         assert r.status_code == 404
430
431
432     def test_process_hosted_mode_rejects_nonlocal_uri(env):
433         """Hosted mode (allowed_video_root set) must reject a bucket/Drive URI ? a tenant
can't point
434         the engine at an arbitrary bucket. Pins the security contract instead of an
incidental side effect."""
435         client, db, tmp_path, mp = env
436         root = tmp_path / "tenant"
437         root.mkdir()
438         mp.setattr(m.global_config.security, "allowed_video_root", str(root))
439         for bad in ("gs://b/x.mp4", "s3://b/x.mp4", "gdrive://Sharing/x.mp4"):
440             r = client.post("/api/process", json={"video_path": bad})
441             assert r.status_code == 400, f"{bad} -> {r.status_code}"
442         # A local file UNDER the root is still accepted (passes validation; processing
mocked elsewhere).
443         under = root / "ok.mp4"
444         under.write_bytes(b"hashable")
445         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
446         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
447         r = client.post("/api/process", json={"video_path": str(under), "segmenter": "x"})
448         assert r.status_code == 202
449
450
451     def test_cli_nonlocal_input_fetches_and_keeps_uri(tmp_path, monkeypatch):
452         """The CLI --video-path block (a second M2 call-site) routes a bucket URI through
fetch: the
```

```
453         segmenter sees a LOCAL path and the DB keeps the source URI."""
454         db = Database(str(tmp_path / "cli.db"))
455         cfg = AppConfig()
456         cfg.output.output_dir = str(tmp_path)
457         monkeypatch.setattr(m, "load_app_config", lambda *a, **k: cfg)
458         monkeypatch.setattr(m, "Database", lambda *a, **k: db)
459         monkeypatch.setattr(m, "global_config", cfg)
460         monkeypatch.setattr(m, "db_instance", db)
461         monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(),
{}))
462         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
463
464         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
465             with open(dst, "wb") as fh:
466                 fh.write(b"cli-video-bytes")
467             return dst
468
469         monkeypatch.setattr(m.storage, "fetch", fake_fetch)
470         _CaptureSeg.last_path = None
471         monkeypatch.setattr(sys, "argv", ["prog", "--video-path",
"gs://bucket/videos/cli.mp4",
472                                           "--output-dir", str(tmp_path)])
473         m.main()
474         assert _CaptureSeg.last_path is not None
475         assert not _CaptureSeg.last_path.startswith("gs://")          # fetched LOCAL path
476         assert os.path.basename(_CaptureSeg.last_path) == "cli.mp4"
477         vid = m.compute_video_id(_CaptureSeg.last_path)
478         rec = db.get_video(vid)
479         assert rec is not None and rec["filepath"] == "gs://bucket/videos/cli.mp4"  # source
URI kept
480
481
482     def test_cli_missing_local_returns_without_segmenting(tmp_path, monkeypatch):
483         """A missing LOCAL --video-path fast-fails (prints + returns) before ever touching
the segmenter."""
484         db = Database(str(tmp_path / "cli2.db"))
485         cfg = AppConfig()
486         cfg.output.output_dir = str(tmp_path)
487         monkeypatch.setattr(m, "load_app_config", lambda *a, **k: cfg)
488         monkeypatch.setattr(m, "Database", lambda *a, **k: db)
489         monkeypatch.setattr(m, "global_config", cfg)
490         monkeypatch.setattr(m, "db_instance", db)
491         called = {"seg": False}
492
493         class _NoSeg:
494             def __init__(self, db, cfg):
495                 pass
496
497             def process_video(self, *a, **k):
498                 called["seg"] = True
499                 return []
500
501         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _NoSeg)
502         monkeypatch.setattr(sys, "argv", ["prog", "--video-path",
503                                           str(tmp_path / "nope.mp4"), "--output-dir",
str(tmp_path)])
504         m.main()
505         assert called["seg"] is False
506
```

5. user (2026-06-21T09:17:08.659Z)

```
1       #!/usr/bin/env python
2       """Run the FULL test suite for the badminton-highlight-indexer.
3
```

```
4       A single, discoverable entry point so you (or a reviewer / CI) can verify the
5       whole suite before opening a PR:
6
7           python run_all_tests.py             # full suite, quiet
8           python run_all_tests.py -v          # verbose (passthrough to pytest)
9           python run_all_tests.py -k config   # filter by keyword (passthrough)
10          python run_all_tests.py --cov=backend --cov-report=term-missing   # coverage
11
12      Anything after the script name is passed straight through to pytest.
13
14      Notes
15      -----
16      - The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
17      - The observability **reporting** tests skip cleanly if the optional ``obsreport``
18        package isn't installed (they use ``pytest.importorskip``). To exercise them too:
19            pip install -e ../sports-obsreport --no-build-isolation
20      - Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
21      """
22      from __future__ import annotations
23
24      import importlib.util
25      import subprocess
26      import sys
27
28
29      def _installed(module: str) -> bool:
30          try:
31              return importlib.util.find_spec(module) is not None
32          except (ImportError, ValueError):
33              return False
34
35
36      def main() -> int:
37          args = list(sys.argv[1:])
38          # Default to quiet unless the caller chose a verbosity.
39          if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
40              args = ["-q", *args]
41          cmd = [sys.executable, "-m", "pytest", *args]
42
43          obs = "available" if _installed("obsreport") else "NOT installed ? reporting tests
will SKIP"
44          bar = "=" * 72
45          print(bar)
46          print(" Full test suite:  " + " ".join(cmd))
47          print(f" obsreport (optional reporting dep): {obs}")
48          print(bar, flush=True)
49
50          rc = subprocess.run(cmd).returncode
51
52          print(bar)
53          print(" RESULT: " + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit
{rc}"))
54          print(bar)
55          return rc
56
57
58      if __name__ == "__main__":
59          raise SystemExit(main())
60
```

6. assistant (2026-06-21T09:17:11.596Z)

I have a clear picture of `test_async_jobs.py` and `test_api_endpoints.py`. Now let me read the remaining test files and check for conftest, plus confirm the `m.storage` seam and `get_storage`/`GCSStorage` injection.

7. user (2026-06-21T09:17:12.545Z)

```
1       """Input acquisition for COMPUTE_DECOUPLED_SERVING M2 ? resolve_input_uri /
input_is_local /
2       acquire_local_input.
3
4       The hard guarantee: the default config (input_backend='local', input_root='') is a byte-
for-byte
5       identity passthrough for every input path used today (no fetch, no copy). A bucket/Drive
source is
6       routed through storage.fetch to a local scratch file. All offline (the remote backend is
a
7       monkeypatched fetch ? no boto3/google-cloud-storage, no network).
8
9       OS note: os.path.isabs is platform-dependent, so the absolute-path cases use a POSIX-
style
10      '/data/...' (absolute on BOTH Windows and POSIX) and Windows backslash paths only with
input_root=''
11      (branch 5, OS-independent) ? the full suite runs cross-OS in CI.
12      """
13      import os
14
15      import pytest
16
17      from backend import storage
18      from backend.storage import acquire_local_input, input_is_local, resolve_input_ref
19      from backend.storage.ref import resolve_input_uri
20
21
22      # ------------------------------------------------------------------ resolve_input_uri
23      @pytest.mark.parametrize("raw,ib,ir,expected", [
24          # 1. explicit scheme always wins (even over a non-local backend + root)
25          ("gs://b/x.mp4", "local", "", "gs://b/x.mp4"),
26          ("s3://b/x.mp4", "gcs", "gs://r", "s3://b/x.mp4"),
27          ("local://abs/x.mp4", "gcs", "gs://r", "local://abs/x.mp4"),
28          ("gdrive://Sharing/x.mp4", "local", "", "gdrive://Sharing/x.mp4"),
29          # 2. an anchored local path (leading slash, or drive prefix) is always local, never
joined ?
30          #    OS-independently (not via os.path.isabs, which differs on Windows+py3.13)
31          ("/data/x.mp4", "gcs", "gs://r", "/data/x.mp4"),
32          ("\\\\server\\share\\x.mp4", "gcs", "gs://r", "\\\\server\\share\\x.mp4"),
33          ("C:\\v\\x.mp4", "gcs", "gs://r", "C:\\v\\x.mp4"),
34          ("C:/v/x.mp4", "gcs", "gs://r", "C:/v/x.mp4"),
35          # 3. a bare/relative name joins under a non-empty input_root
36          ("x.mp4", "local", "gs://bucket/videos", "gs://bucket/videos/x.mp4"),
37          ("sub/x.mp4", "local", "gs://bucket/videos/", "gs://bucket/videos/sub/x.mp4"),  #
root trailing slash
38          # 4. input_backend prefixes a bare bucket/key when no input_root
39          ("bucket/x.mp4", "gcs", "", "gcs://bucket/x.mp4"),
40          ("bucket/x.mp4", "s3", "", "s3://bucket/x.mp4"),
41          # 5. bare/relative local path ? today's behaviour, unchanged
42          ("x.mp4", "local", "", "x.mp4"),
43          ("rel/sub/x.mp4", "local", "", "rel/sub/x.mp4"),
44          ("C:\\v\\x.mp4", "local", "", "C:\\v\\x.mp4"),  # Windows path, default config:
unchanged (OS-independent)
45      ])
46      def test_resolve_input_uri(raw, ib, ir, expected):
47          assert resolve_input_uri(raw, input_backend=ib, input_root=ir) == expected
48
49
50      def test_resolve_input_uri_default_is_identity():
51          for p in ("/data/clip.mp4", "rel/clip.mp4", "clip.mp4", "gs://b/k", "local://x"):
52              assert resolve_input_uri(p) == p
53
54
```

```
    # ------------------------------------------------------------- input_is_local
    def test_input_is_local_true_for_local_paths():
        assert input_is_local("/data/x.mp4") is True
        assert input_is_local("rel/x.mp4") is True
        assert input_is_local("local://x.mp4") is True


    def test_input_is_local_false_for_buckets():
        assert input_is_local("gs://b/x.mp4") is False
        assert input_is_local("s3://b/x.mp4") is False
        assert input_is_local("gdrive://Sharing/x.mp4") is False


    def test_input_is_local_follows_input_root():
        cfg = {"input_backend": "local", "input_root": "gs://bucket/videos"}
        assert input_is_local("x.mp4", cfg) is False           # bare name resolves into the
bucket
        assert input_is_local("/abs/x.mp4", cfg) is True        # absolute path stays local
        assert input_is_local("gs://other/x.mp4", cfg) is False


    def test_input_is_local_accepts_pydantic_storage_config():
        from backend.config.models import StorageConfig
        sc = StorageConfig(input_root="gs://bucket/videos")
        assert input_is_local("x.mp4", sc) is False
        assert input_is_local("/abs/x.mp4", sc) is True


    def test_input_is_local_unknown_scheme_raises():
        # An unsupported scheme is a ValueError (callers at the API/CLI boundary map it to a
clean error),
        # never a silent True/False that could mislead a security gate.
        for bad in ("https://h/x.mp4", "ftp://h/x.mp4", "file:///etc/passwd"):
            with pytest.raises(ValueError):
                input_is_local(bad)


    # ------------------------------------------------------------- resolve_input_ref
    def test_resolve_input_ref_local_and_remote():
        local = resolve_input_ref("/data/x.mp4")
        assert (local.backend, local.key) == ("local", "/data/x.mp4")
        # local:// strips to the bare key (so callers stat ref.key, not the raw URI)
        assert resolve_input_ref("local:///data/x.mp4").key == "/data/x.mp4"
        remote = resolve_input_ref("x.mp4", {"input_root": "gs://bucket/videos"})
        assert (remote.backend, remote.bucket, remote.key) == ("gcs", "bucket",
"videos/x.mp4")


    def test_resolve_input_ref_unknown_scheme_raises():
        with pytest.raises(ValueError):
            resolve_input_ref("https://host/x.mp4")


    # ------------------------------------------------------------- acquire_local_input
    def test_acquire_local_input_local_identity(tmp_path):
        p = tmp_path / "v.mp4"
        p.write_bytes(b"abc")
        # Identity passthrough ? returns the SAME string, no copy, no temp dir.
        assert acquire_local_input(str(p)) == str(p)


    def test_acquire_local_input_relative_local_identity():
        assert acquire_local_input("rel/clip.mp4") == "rel/clip.mp4"


```

```
117     def _capturing_fetch(captured):
118         def _f(uri, dst=None, *, config=None, overwrite=False):
119             captured["uri"] = uri
120             captured["dst"] = dst
121             captured["config"] = config
122             if dst:
123                 with open(dst, "wb") as fh:
124                     fh.write(b"remote-bytes")
125             return dst or uri
126         return _f
127
128
129     def test_acquire_local_input_remote_fetches_to_scratch(monkeypatch):
130         captured = {}
131         monkeypatch.setattr(storage, "fetch", _capturing_fetch(captured))
132         out = acquire_local_input("gs://bucket/videos/x.mp4", {"backend": "local", "gcs":
{}})
133         assert captured["uri"] == "gs://bucket/videos/x.mp4"          # exact URI routed to
fetch
134         assert os.path.basename(out) == "x.mp4"                       # named from the ref
135         assert os.path.isfile(out) and open(out, "rb").read() == b"remote-bytes"
136         # The whole storage settings dict is forwarded so per-backend (gcs/s3) config
reaches fetch.
137         assert captured["config"] == {"backend": "local", "gcs": {}}
138
139
140     def test_acquire_local_input_input_root_joins_then_fetches(monkeypatch):
141         captured = {}
142         monkeypatch.setattr(storage, "fetch", _capturing_fetch(captured))
143         acquire_local_input("x.mp4", {"input_root": "gs://bucket/videos"})
144         assert captured["uri"] == "gs://bucket/videos/x.mp4"          # resolved BEFORE
fetch
145
146
147     def test_acquire_local_input_never_fetches_for_local(monkeypatch):
148         def _boom(*a, **k):
149             raise AssertionError("fetch must not be called for a local input")
150         monkeypatch.setattr(storage, "fetch", _boom)
151         assert acquire_local_input("/data/x.mp4") == "/data/x.mp4"
152         assert acquire_local_input("clip.mp4", {"input_backend": "local", "input_root": ""})
== "clip.mp4"
153
```

8. user (2026-06-21T09:17:13.020Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       The S3/GCS backends are exercised with injected FAKE clients (no boto3/google-cloud-
storage,
4       no network); the gdrive backend reads a locally MOUNTED Drive, exercised against a
tmp_path
5       that stands in for the mount ? all offline, matching the repo's ethos.
6       """
7       import os
8       import subprocess
9       import sys
10
11      import pytest
12
13      from backend import storage
14      from backend.storage import StorageBackend, fetch, get_storage, list_uris, parse_uri,
put
15      from backend.storage.local import LocalStorage
16      from backend.storage.s3 import S3Storage
17      from backend.storage.gcs import GCSStorage
```

```
    from backend.storage.gdrive import GDriveStorage


    # ------------------------------------------------------------- URI parsing

    @pytest.mark.parametrize("uri,backend,bucket,key", [
        ("/data/x.mp4", "local", "", "/data/x.mp4"),
        ("rel/x.mp4", "local", "", "rel/x.mp4"),
        ("C:/Users/a/x.mp4", "local", "", "C:/Users/a/x.mp4"),       # Windows path: no
':://'
        ("C:\\Users\\a\\x.mp4", "local", "", "C:\\Users\\a\\x.mp4"),
        ("local:///abs/x.mp4", "local", "", "/abs/x.mp4"),
        ("s3://rally/videos/m.mp4", "s3", "rally", "videos/m.mp4"),
        ("gs://rally-eu/w/model.json", "gcs", "rally-eu", "w/model.json"),   # gs:// -> gcs
        ("gcs://rally/x", "gcs", "rally", "x"),
        ("gdrive://Sharing/clip.mp4", "gdrive", "", "Sharing/clip.mp4"),     # path under My
Drive (no bucket)
    ])
    def test_parse_uri(uri, backend, bucket, key):
        ref = parse_uri(uri)
        assert (ref.backend, ref.bucket, ref.key) == (backend, bucket, key)


    def test_parse_uri_rejects_unknown_scheme():
        with pytest.raises(ValueError):
            parse_uri("ftp://host/x")


    # ------------------------------------------------------------ local backend

    def test_local_fetch_is_identity_passthrough(tmp_path):
        src = tmp_path / "v.mp4"
        src.write_bytes(b"abc")
        # No dst => returns the source path unchanged (no copy): preserves today's
behaviour.
        assert fetch(str(src)) == str(src)


    def test_local_put_and_roundtrip(tmp_path):
        src = tmp_path / "in.csv"
        src.write_text("Frame,Visibility,X,Y\n")
        dst_uri = str(tmp_path / "out" / "copy.csv")
        put(str(src), dst_uri)
        assert (tmp_path / "out" / "copy.csv").read_text() == src.read_text()


    def test_local_exists_stat_list_delete(tmp_path):
        be = LocalStorage()
        (tmp_path / "a.txt").write_text("hello")
        (tmp_path / "sub").mkdir()
        (tmp_path / "sub" / "b.txt").write_text("hi")
        assert be.exists(parse_uri(str(tmp_path / "a.txt")))
        assert not be.exists(parse_uri(str(tmp_path / "nope.txt")))
        assert be.stat(parse_uri(str(tmp_path / "a.txt"))).size == 5
        listed = sorted(r.key for r in be.list(parse_uri(str(tmp_path))))
        assert listed == sorted([str(tmp_path / "a.txt"), str(tmp_path / "sub" / "b.txt")])
        be.delete(parse_uri(str(tmp_path / "a.txt")))
        assert not (tmp_path / "a.txt").exists()


    def test_local_signed_url_raises():
        with pytest.raises(NotImplementedError):
            LocalStorage().signed_url(parse_uri("/x"))


```

```
    def test_list_uris_helper(tmp_path):
        (tmp_path / "x.txt").write_text("1")
        (tmp_path / "y.txt").write_text("2")
        uris = sorted(list_uris(str(tmp_path)))
        assert all(u.startswith("local://") for u in uris)
        assert len(uris) == 2


    # ------------------------------------------------------------- registry / get_storage

    def test_get_storage_defaults_to_local():
        assert isinstance(get_storage(), LocalStorage)
        assert isinstance(get_storage(config={"backend": "local"}), LocalStorage)


    def test_get_storage_unknown_backend_raises():
        with pytest.raises(ValueError):
            get_storage("azure")


    def test_gdrive_not_mounted_raises_with_remedy(monkeypatch):
        # The scheme parses; if Drive isn't mounted (no auto-detected/configured root)
construction
        # is a clear, actionable error ? install link + how to include the folder in syncing
+ the
        # mount-root override ? rather than a cryptic failure downstream.
        monkeypatch.delenv("GDRIVE_MOUNT_ROOT", raising=False)
        assert parse_uri("gdrive://Sharing/x.mp4").backend == "gdrive"
        with pytest.raises(RuntimeError) as ei:
            # An explicit, non-existent root forces the error regardless of any real local
mount.
            get_storage("gdrive", config={"gdrive": {"mount_root": "/no-such/Drive/My
Drive"}})
        msg = str(ei.value)
        assert "drive.google.com" in msg or "google.com/drive" in msg   # install pointer
        assert "GDRIVE_MOUNT_ROOT" in msg                               # how to point at
the mount
        assert "shortcut" in msg.lower() and "stream" in msg.lower()    # how to sync the
folder


    def test_gdrive_mounted_but_unsynced_path_raises_with_remedy(tmp_path):
        # Mount exists but the requested file isn't there yet (folder not synced) -> a
FileNotFoundError
        # that names the fix (include the folder in syncing) instead of a bare "No such
file".
        be = GDriveStorage(mount_root=str(_mount(tmp_path)))
        with pytest.raises(FileNotFoundError) as ei:
            be.fetch_to_local(parse_uri("gdrive://Sharing/not-synced.mp4"), str(tmp_path /
"out.mp4"))
        msg = str(ei.value).lower()
        assert "not present locally" in msg and "shortcut" in msg and "stream" in msg


    # ------------------------------------------------------------- S3 (fake client)

    class _FakeS3Client:
        def __init__(self):
            self.store = {}

        def upload_file(self, src, bucket, key, ExtraArgs=None):
            with open(src, "rb") as f:
                self.store[(bucket, key)] = f.read()

        def download_file(self, bucket, key, dst):
```

```
            with open(dst, "wb") as f:
                f.write(self.store[(bucket, key)])

        def head_object(self, Bucket, Key):
            if (Bucket, Key) not in self.store:
                raise KeyError("404")
            return {"ContentLength": len(self.store[(Bucket, Key)]), "ETag": '"e"',
                    "ContentType": "video/mp4"}

        def get_paginator(self, _op):
            store = self.store

            class _P:
                def paginate(self, Bucket, Prefix=""):
                    yield {"Contents": [{"Key": k} for (b, k) in store
                                        if b == Bucket and k.startswith(Prefix)]}
            return _P()

        def delete_object(self, Bucket, Key):
            self.store.pop((Bucket, Key), None)

        def generate_presigned_url(self, op, Params, ExpiresIn):
            return f"https://s3.example/{Params['Bucket']}/{Params['Key']}?X-Amz-
Expires={ExpiresIn}"


    def test_s3_backend_roundtrip_with_fake_client(tmp_path):
        be = S3Storage(client=_FakeS3Client())
        assert isinstance(be, StorageBackend)
        src = tmp_path / "m.mp4"
        src.write_bytes(b"VIDEO")
        ref = parse_uri("s3://rally/videos/m.mp4")
        be.put(str(src), ref, content_type="video/mp4")
        assert be.exists(ref)
        assert be.stat(ref).size == 5
        dst = tmp_path / "dl" / "m.mp4"
        assert be.fetch_to_local(ref, str(dst)) == str(dst)
        assert dst.read_bytes() == b"VIDEO"
        assert [r.uri for r in be.list(parse_uri("s3://rally/videos/"))] ==
["s3://rally/videos/m.mp4"]
        url = be.signed_url(ref, ttl=900)
        assert url.startswith("https://s3.example/rally/videos/m.mp4") and "900" in url
        be.delete(ref)
        assert not be.exists(ref)


    def test_s3_exists_false_on_missing():
        assert S3Storage(client=_FakeS3Client()).exists(parse_uri("s3://b/none")) is False


    # ---------------------------------------------------------------- GCS (fake client)

    class _FakeBlob:
        def __init__(self, store, bucket, name):
            self.store, self.bucket, self.name = store, bucket, name
            self.size = self.updated = self.etag = self.content_type = None

        def upload_from_filename(self, src, content_type=None):
            with open(src, "rb") as f:
                self.store[(self.bucket, self.name)] = f.read()
            self.content_type = content_type

        def download_to_filename(self, dst):
            with open(dst, "wb") as f:
                f.write(self.store[(self.bucket, self.name)])
```

```
199
200         def exists(self):
201             return (self.bucket, self.name) in self.store
202
203         def reload(self):
204             self.size = len(self.store[(self.bucket, self.name)])
205
206         def delete(self):
207             self.store.pop((self.bucket, self.name), None)
208
209         def generate_signed_url(self, version, expiration, method):
210             return f"https://gcs.example/{self.bucket}/{self.name}?version={version}"
211
212
213     class _FakeGCSClient:
214         def __init__(self):
215             self.store = {}
216
217         def bucket(self, name):
218             return type("_B", (), {"blob": lambda _self, key: _FakeBlob(self.store, name,
key)})()
219
220         def list_blobs(self, bucket, prefix=""):
221             return [_FakeBlob(self.store, bucket, k) for (b, k) in self.store
222                     if b == bucket and k.startswith(prefix)]
223
224
225     def test_gcs_backend_roundtrip_with_fake_client(tmp_path):
226         be = GCSStorage(client=_FakeGCSClient())
227         src = tmp_path / "w.json"
228         src.write_text("{}")
229         ref = parse_uri("gcs://rally/weights/w.json")
230         be.put(str(src), ref)
231         assert be.exists(ref)
232         assert be.stat(ref).size == 2
233         dst = tmp_path / "dl" / "w.json"
234         be.fetch_to_local(ref, str(dst))
235         assert dst.read_text() == "{}"
236         assert [r.uri for r in be.list(parse_uri("gcs://rally/weights/"))] ==
["gcs://rally/weights/w.json"]
237         assert be.signed_url(ref).startswith("https://gcs.example/rally/weights/w.json")
238         be.delete(ref)
239         assert not be.exists(ref)
240
241
242     # ------------------------------------------------------------- Google Drive (local
mount)
243     # The backend reads a locally MOUNTED Google Drive (Google Drive for Desktop). A
tmp_path
244     # folder stands in for the mount's "My Drive" root ? no network, no Drive API, no extra
deps.
245
246     def _mount(tmp_path):
247         root = tmp_path / "My Drive"
248         (root / "Sharing").mkdir(parents=True)
249         return root
250
251
252     def test_gdrive_backend_roundtrip_over_local_mount(tmp_path):
253         root = _mount(tmp_path)
254         be = GDriveStorage(mount_root=str(root))
255         assert isinstance(be, StorageBackend)
256         # A file present in the mount, exactly as Drive for Desktop would surface it.
257         (root / "Sharing" / "clip.mp4").write_bytes(b"VIDEO")
258         ref = parse_uri("gdrive://Sharing/clip.mp4")
```

```
        assert be.exists(ref)
        assert be.stat(ref).size == 5
        # No dst => identity passthrough to the on-mount file (no needless copy).
        assert os.path.samefile(be.fetch_to_local(ref), root / "Sharing" / "clip.mp4")
        # A dst => copy out (byte-stable scratch copy), like the local backend.
        dst = tmp_path / "scratch" / "in.mp4"
        assert be.fetch_to_local(ref, str(dst)) == str(dst)
        assert dst.read_bytes() == b"VIDEO"
        # list a folder -> gdrive:// URIs relative to the mount; `name` is the basename.
        assert [r.uri for r in be.list(parse_uri("gdrive://Sharing"))] ==
["gdrive://Sharing/clip.mp4"]
        assert [r.name for r in be.list(parse_uri("gdrive://Sharing"))] == ["clip.mp4"]
        be.delete(ref)
        assert not be.exists(ref)


    def test_gdrive_put_writes_into_the_mount(tmp_path):
        root = _mount(tmp_path)
        src = tmp_path / "out.mp4"
        src.write_bytes(b"REEL")
        ret = GDriveStorage(mount_root=str(root)).put(str(src),
parse_uri("gdrive://Sharing/out.mp4"))
        assert ret.uri == "gdrive://Sharing/out.mp4"
        assert (root / "Sharing" / "out.mp4").read_bytes() == b"REEL"


    def test_gdrive_mount_root_from_env(tmp_path, monkeypatch):
        root = _mount(tmp_path)
        (root / "v.mp4").write_bytes(b"X")
        monkeypatch.setenv("GDRIVE_MOUNT_ROOT", str(root))
        assert GDriveStorage().exists(parse_uri("gdrive://v.mp4"))         # env supplies
the root


    def test_gdrive_autodetect_is_safe_across_platforms():
        # Platform-dependent (a real mount or None), so assert only the invariant: it never
crashes
        # and any hit ends at a 'My Drive' folder. Covers the detect logic on both win/posix
lanes.
        from backend.storage.gdrive import _autodetect_mount_root
        root = _autodetect_mount_root()
        assert root is None or root.endswith("My Drive")


    def test_gdrive_exists_false_on_missing(tmp_path):
        assert
GDriveStorage(mount_root=str(_mount(tmp_path))).exists(parse_uri("gdrive://nope.mp4")) is False


    def test_gdrive_signed_url_not_supported(tmp_path):
        # A mounted Drive file has no presigned URL; callers fall back to fetch_to_local.
        with pytest.raises(NotImplementedError):

GDriveStorage(mount_root=str(_mount(tmp_path))).signed_url(parse_uri("gdrive://v.mp4"))


    # ------------------------------------------------------------------ lazy-import hygiene

    def test_importing_storage_does_not_pull_cloud_sdks():
        # `import backend.storage` must not eagerly import boto3 / google-cloud-storage, so
the
        # default install (and the CI lanes without those extras) stays clean.
        code = (
            "import sys, backend.storage; "
            "assert 'boto3' not in sys.modules, 'boto3 imported eagerly'; "
```

```
316             "assert 'google.cloud.storage' not in sys.modules, 'gcs imported eagerly'; "
317             "print('clean')"
318         )
319         r = subprocess.run([sys.executable, "-c", code], capture_output=True, text=True)
320         assert r.returncode == 0, r.stderr
321         assert "clean" in r.stdout
322
323
324     def test_config_storage_defaults_are_non_breaking():
325         from backend.config import StorageConfig
326         sc = StorageConfig()
327         assert sc.backend == "local" and sc.output_root == ""
328         assert isinstance(storage.get_storage(config=sc.model_dump()), LocalStorage)
329
```

9. user (2026-06-21T09:17:13.959Z)

```
1       """Characterization pins for the job-worker drain/wake seam (refactor guard).
2
3       This file PINS the exact indirection contract that made the
4       `backend.api.job_worker` extraction risky to do blindly: the worker's
5       inter-function calls must resolve the monkeypatchable seams
6       (`_arm_wake_timer`, `_get_executor`, `db_instance`, `_execute_processing`)
7       THROUGH the `backend.main` module object, so that `monkeypatch.setattr(m, ...)`
8       (as test_job_worker_loop.py / test_async_jobs.py do) still intercepts and no
9       real daemon timer leaks.
10
11      Written against the CURRENT behavior and confirmed green BEFORE the extraction;
12      must stay green, unchanged, after it.
13      """
14      import threading
15
16      import backend.main as m
17
18
19      def test_symbols_are_attributes_of_backend_main():
20          """The worker symbols the existing tests reference via `m.<name>` must remain
21          attributes of `backend.main` (re-export shim keeps the names addressable)."""
22          for name in (
23              "JobWaiting",
24              "_get_executor",
25              "_job_to_request",
26              "_run_claimed_job",
27              "_drain_due_jobs",
28              "_schedule_next_drain_if_waiting",
29              "_arm_wake_timer",
30              "_MAX_DRAIN_WAKE_SEC",
31          ):
32              assert hasattr(m, name), f"backend.main must expose {name}"
33          assert m._MAX_DRAIN_WAKE_SEC == 60.0
34
35
36      def test_drain_routes_wake_through_patched_backend_main_arm_timer(tmp_path,
monkeypatch):
37          """THE HAZARD PIN: a drain that finds a not-yet-due 'waiting' job must arm the
38          wake-timer THROUGH `backend.main._arm_wake_timer`, so patching it on
39          `backend.main` intercepts and no live daemon timer is created. Patch
40          `m._arm_wake_timer` (the documented seam), run a real drain, and assert it was
41          the patched recorder that fired ? not a bare local that would leak a timer."""
42          from backend.infrastructure.database import Database
43
44          db = Database(str(tmp_path / "t.db"))
45          monkeypatch.setattr(m, "db_instance", db)
46          armed: list = []
47          monkeypatch.setattr(m, "_arm_wake_timer", lambda delay: armed.append(delay))
```

```
48
49          db.create_job("j1", "/v.mp4")
50          db.set_job_waiting("j1", delay_sec=3600, progress="parked")
51
52          ran = m._drain_due_jobs()
53
54          assert ran == 0  # not due -> not claimed
55          assert db.get_job("j1")["status"] == "waiting"
56          # The patched seam fired (clamped to the cap) ? proves the indirection holds.
57          assert armed and 0.0 < armed[-1] <= m._MAX_DRAIN_WAKE_SEC
58
59
60      def test_arm_wake_timer_submits_drain_through_patched_executor(monkeypatch):
61          """`_arm_wake_timer` must submit the re-drain through `backend.main._get_executor`
62          and `backend.main._drain_due_jobs`, so a stubbed executor + a real (non-leaking)
63          timer let us observe the scheduled callback without sleeping or leaking threads."""
64          submitted: list = []
65
66          class _RecordingExecutor:
67              def submit(self, fn, *a, **k):
68                  submitted.append((fn, a, k))
69
70          monkeypatch.setattr(m, "_get_executor", lambda: _RecordingExecutor())
71
72          captured = {}
73
74          class _FakeTimer:
75              def __init__(self, delay, fn):
76                  captured["delay"] = delay
77                  captured["fn"] = fn
78                  self.daemon = False
79
80              def start(self):
81                  captured["started"] = True
82                  self.fn_result = self.fn()  # fire immediately, no real wait
83
84              @property
85              def fn(self):
86                  return captured["fn"]
87
88          # Patch on the shared `threading` module object itself (the module-agnostic seam) so
89          # this pin doesn't couple to which module `_arm_wake_timer` lives in.
90          monkeypatch.setattr(threading, "Timer", _FakeTimer)
91
92          m._arm_wake_timer(2.5)
93
94          assert captured["delay"] == 2.5
95          assert captured.get("started") is True
96          # The timer callback submitted a drain via the PATCHED executor.
97          assert submitted and submitted[0][0] is m._drain_due_jobs
98
```

10. user (2026-06-21T09:17:14.084Z)

```
1       """Tests for the v3 self-scheduling job substrate (EXECUTION_POLICY.md P2).
2
3       Covers the policy column round-trip, set_job_waiting, and the atomic claim_due_job
4       (queued + due-waiting claimable, not-yet-due skipped, drained-then-None, compare-and-
set).
5       """
6       from backend.infrastructure.database import Database
7
8       POLICY = {"mode": "wait_and_resume", "op_timeout_sec": 90.0, "max_retries": 3}
9
10
```

```
    def _db(tmp_path):
        return Database(str(tmp_path / "jobs.db"))


    def test_policy_roundtrips_through_create_and_get(tmp_path):
        db = _db(tmp_path)
        assert db.create_job("j1", "/v.mp4", policy=POLICY)
        assert db.get_job("j1")["policy"] == POLICY


    def test_create_without_policy_is_none(tmp_path):
        db = _db(tmp_path)
        db.create_job("j0", "/v.mp4")
        assert db.get_job("j0")["policy"] is None


    def test_set_job_policy_replaces(tmp_path):
        db = _db(tmp_path)
        db.create_job("j1", "/v.mp4")
        assert db.set_job_policy("j1", {"mode": "abort"})
        assert db.get_job("j1")["policy"] == {"mode": "abort"}


    def test_set_job_waiting_sets_status_and_due(tmp_path):
        db = _db(tmp_path)
        db.create_job("j1", "/v.mp4")
        assert db.set_job_waiting("j1", delay_sec=3600, progress="waiting on gemini")
        row = db.get_job("j1")
        assert row["status"] == "waiting" and row["next_poll_at"] is not None
        assert row["progress"] == "waiting on gemini"


    def test_claim_due_job_none_when_empty(tmp_path):
        assert _db(tmp_path).claim_due_job() is None


    def test_claim_due_job_claims_queued_and_flips_running(tmp_path):
        db = _db(tmp_path)
        db.create_job("j1", "/v.mp4", policy=POLICY)
        claimed = db.claim_due_job()
        assert claimed["id"] == "j1" and claimed["status"] == "running"
        assert claimed["policy"] == POLICY                      # policy travels with the
claim
        assert db.get_job("j1")["status"] == "running"


    def test_claim_drains_then_returns_none(tmp_path):
        db = _db(tmp_path)
        db.create_job("a", "/a.mp4")
        db.create_job("b", "/b.mp4")
        got = {db.claim_due_job()["id"], db.claim_due_job()["id"]}
        assert got == {"a", "b"}                                # both drained (order not
asserted)
        assert db.claim_due_job() is None                       # nothing runnable left


    def test_waiting_job_due_is_claimable(tmp_path):
        db = _db(tmp_path)
        db.create_job("j1", "/v.mp4")
        db.set_job_waiting("j1", delay_sec=0)                   # due immediately
        claimed = db.claim_due_job()
        assert claimed is not None and claimed["id"] == "j1" and claimed["status"] ==
"running"


```

```
73      def test_waiting_job_not_due_is_skipped(tmp_path):
74          db = _db(tmp_path)
75          db.create_job("j1", "/v.mp4")
76          db.set_job_waiting("j1", delay_sec=3600)                # due in an hour
77          assert db.claim_due_job() is None                       # not yet due -> not
claimed
78          assert db.get_job("j1")["status"] == "waiting"          # left parked
79
80
81      def test_running_job_is_not_reclaimed(tmp_path):
82          db = _db(tmp_path)
83          db.create_job("j1", "/v.mp4")
84          db.claim_due_job()                                      # -> running
85          assert db.claim_due_job() is None                       # a running job is not
runnable
86
```

11. assistant (2026-06-21T09:17:17.666Z)

Now let me check the #281 guard specifically (the double-module subprocess hang guard), look for conftest.py, and confirm the `m.storage` / `get_storage` seam in `backend/main.py` and `backend/storage`.

12. user (2026-06-21T09:17:18.387Z)

No files found

13. user (2026-06-21T09:17:18.777Z)

tests\test_job_worker_loop.py
tests\test_job_worker_module_seam.py

14. user (2026-06-21T09:17:19.860Z)

Found 12 files
tests\test_native_wasb_runner.py
tests\test_server_entrypoint.py
tests\test_api_endpoints.py
tests\test_worker.py
tests\test_training_gen0.py
tests\test_storage.py
tests\test_tracknet_runner.py
tests\test_compiler.py
tests\test_run_telemetry.py
tests\test_watermark.py
tests\test_video.py
tests\test_import_smoke.py

15. assistant (2026-06-21T09:17:24.165Z)

The #281 guard is in `test_server_entrypoint.py` (mentions "double-module" / `python -m backend.main`). Let me read it, plus check how `m.storage` and `acquire_local_input` are wired in `backend/main.py` to confirm the exact fetch seam.

16. user (2026-06-21T09:17:24.877Z)

```
1       """Regression guard for the `python -m backend.main` double-module job-hang.
2
3       `python -m backend.main` runs main.py as `__main__`, so `main()` configures `__main__`'s
globals
4       (`db_instance`/`global_config`). But `job_worker`/`uploads` resolve shared state via
5       `import backend.main` ? a SEPARATE module object whose `db_instance` stays `None`. The
async-job
6       drain then crashes silently on `None.claim_due_job()` and every submitted job hangs in
```

```
`queued`
7       forever. The in-process tests (TestClient + monkeypatched `db_instance` + an inline
executor)
8       cannot see this ? only the real subprocess does. This spawns the DOCUMENTED invocation
and asserts
9       a submitted job actually LEAVES `queued`.
10      """
11      import json
12      import os
13      import socket
14      import subprocess
15      import sys
16      import time
17      import urllib.error
18      import urllib.request
19
20      import pytest
21
22      REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
23
24
25      def _free_port() -> int:
26          s = socket.socket()
27          s.bind(("127.0.0.1", 0))
28          port = int(s.getsockname()[1])
29          s.close()
30          return port
31
32
33      def _get(url: str, timeout: float = 5):
34          with urllib.request.urlopen(url, timeout=timeout) as r:
35              return json.loads(r.read().decode())
36
37
38      def _post(url: str, payload: dict, timeout: float = 15):
39          req = urllib.request.Request(
40              url, data=json.dumps(payload).encode(), headers={"content-type":
"application/json"})
41          with urllib.request.urlopen(req, timeout=timeout) as r:
42              return json.loads(r.read().decode())
43
44
45      def test_python_m_backend_main_drains_submitted_jobs(tmp_path):
46          """Spawning the engine the documented way (`python -m backend.main --server`) must
actually
47          DRAIN a submitted job ? it must leave `queued`, never hang. (A dummy file fails
fast; the
48          verdict is irrelevant ? what matters is that the worker claimed and ran it.)"""
49          port = _free_port()
50          out_dir = tmp_path / "out"
51          video = tmp_path / "dummy.mp4"
52          video.write_bytes(b"not-a-real-video")  # exists -> the drain claims it; processing
fails fast
53          log = open(tmp_path / "server.log", "wb")
54
55          proc = subprocess.Popen(
56              [sys.executable, "-m", "backend.main", "--server", "--port", str(port),
57               "--output-dir", str(out_dir)],
58              cwd=REPO_ROOT, stdout=log, stderr=subprocess.STDOUT,
59          )
60          base = f"http://127.0.0.1:{port}"
61          try:
62              deadline = time.time() + 90  # cold import of torch/cv2 can be slow on CI
63              while time.time() < deadline:
64                  if proc.poll() is not None:
```

```
65                      pytest.fail(f"server exited early (code {proc.returncode}); see
{tmp_path/'server.log'}")
66                  try:
67                      if _get(f"{base}/api/health").get("status") == "ok":
68                          break
69                  except (urllib.error.URLError, ConnectionError, OSError):
70                      time.sleep(0.5)
71              else:
72                  pytest.fail("server never became healthy")
73
74              job_id = _post(f"{base}/api/process",
75                             {"video_path": str(video), "segmenter": "motion"})["job_id"]
76
77              status = "queued"
78              deadline = time.time() + 45
79              while time.time() < deadline:
80                  status = _get(f"{base}/api/jobs/{job_id}")["status"]
81                  if status != "queued":
82                      break
83                  time.sleep(1)
84
85              assert status != "queued", (
86                  "job stayed 'queued' ? the async drain never ran under `python -m
backend.main` "
87                  "(the __main__/backend.main double-module entrypoint bug regressed)."
88              )
89          finally:
90              proc.terminate()
91              try:
92                  proc.wait(timeout=10)
93              except subprocess.TimeoutExpired:
94                  proc.kill()
95              log.close()
96
```

17. user (2026-06-21T09:17:26.685Z)

```
35:from backend import storage  # M2: URI-aware input acquisition (local = identity passthrough)
159:    local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
274:        input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
600:            input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
607:        local_video = storage.acquire_local_input(args.video_path, storage_cfg)
```

18. assistant (2026-06-21T09:17:30.105Z)

I now have the full picture. Let me confirm the exact fetch seam inside `acquire_local_input` (whether it calls module-level `storage.fetch` or `get_storage`) to pin the cleanest hermetic mock, and verify the `/api/process` worker flow around line 159.

19. user (2026-06-21T09:17:31.552Z)

```
150         (after this function has already restored the pre-run segments, see below).
151         """
152         notify = progress or (lambda stage: None)
153
154         notify("hashing")
155         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
156         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
157         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
```

```
158         # consumers (hash / validators / segmenter) use the resolved local path.
159         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
160         video_id = compute_video_id(local_video)
161         was_reingest = db_instance.get_video(video_id) is not None
162
163         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
164         notify("validating")
165         logger.info(f"Running validations for {req.video_path}...")
166         val_results, val_context = registry.run_all_with_context(local_video, global_config)
167         failures = [r for r in val_results if not r.passed]
168
169         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
170         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
171         seg_name = req.segmenter or default_seg
172         segmenter_actual = None
173         segmentation_ran = False
174         response: Optional[Dict[str, Any]] = None
175         try:
176             if failures:
177                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
178                 # status; it no longer destroys the video's prior good segments.
179                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
180                 response = {
181                     "video_id": video_id,
182                     "status": "failed",
183                     "message": "Video failed validation checks.",
184                     "results": [r.model_dump() for r in val_results],
185                 }
186                 return response
187
188             # 2. Run segmenter & indexer
189             logger.info("[API] Video passed checks. Segmenting and indexing...")
```

20. user (2026-06-21T09:17:32.705Z)

```
backend\storage\gcs.py:30:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gcs.py:32:            raise ValueError("GCSStorage.fetch_to_local requires a dst
path")
backend\storage\base.py:18:    """Backend-agnostic object storage. ``fetch_to_local`` + ``put``
are the two the
backend\storage\base.py:22:    def fetch_to_local(self, ref: StorageRef, dst: "str |
os.PathLike[str] | None" = None,
backend\storage\base.py:48:        ``NotImplementedError`` as expected and fall back to
``fetch_to_local``."""
backend\storage\s3.py:38:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
backend\storage\s3.py:40:            raise ValueError("S3Storage.fetch_to_local requires a dst
path")
backend\storage\__init__.py:59:def acquire_local_input(raw: str, storage_config: Any = None, *,
backend\storage\__init__.py:74:    return fetch(ref.uri, os.path.join(scratch, ref.name or
"in.mp4"), config=sc)
backend\storage\__init__.py:94:def get_storage(backend: Optional[str] = None, *,
backend\storage\__init__.py:106:def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None,
*,
backend\storage\__init__.py:111:    return get_storage(ref.backend,
config=config).fetch_to_local(ref, dst, overwrite=overwrite)
backend\storage\__init__.py:118:    get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
backend\storage\__init__.py:125:    for r in get_storage(ref.backend, config=config).list(ref):
backend\storage\local.py:3:Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an
identity passthrough (the
```

```
backend\storage\local.py:29:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gdrive.py:112:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gdrive.py:116:        return self._local.fetch_to_local(self._local_ref(ref),
dst, overwrite=overwrite)
```

21. assistant (2026-06-21T09:17:37.488Z)

I have everything needed. Let me confirm the seam chain in `backend/storage/__init__.py` lines 59-111: `acquire_local_input` calls module-level `fetch`, which calls `get_storage(...).fetch_to_local`. The existing test `test_process_nonlocal_uri_skips_exists_and_fetches` already monkeypatches `m.storage.fetch` ? but `acquire_local_input` calls the bare `fetch` name inside the `backend.storage` module, so the seam that test patches is `m.storage.fetch` (the module attribute). Let me verify the call inside `acquire_local_input` references the module-local `fetch` vs an imported binding.

22. user (2026-06-21T09:17:38.646Z)

```
59      def acquire_local_input(raw: str, storage_config: Any = None, *,
60                              scratch_parent: "str | None" = None) -> str:
61          """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
62
63          Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64          byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
65          non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
66          path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67          regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
68          worker's ``cmd_infer`` already ships."""
69          sc = _storage_dict(storage_config)
70          ref = resolve_input_ref(raw, sc)
71          if ref.backend == "local":
72              return ref.key  # identity ? no temp, no copy, byte-identical to passing the
path directly
73          scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74          return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
75
76
77      def _backend_class(name: str):
78          """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79          if name == "local":
80              from backend.storage.local import LocalStorage
81              return LocalStorage
82          if name == "s3":
83              from backend.storage.s3 import S3Storage
84              return S3Storage
85          if name == "gcs":
86              from backend.storage.gcs import GCSStorage
87              return GCSStorage
88          if name == "gdrive":
89              from backend.storage.gdrive import GDriveStorage
90              return GDriveStorage
91          raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
92
93
94      def get_storage(backend: Optional[str] = None, *,
95                      config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96          """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
```

```
selects
97          the implementation; its per-backend settings come from ``config[backend]`` merged
with
98          ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
99          cfg = config or {}
100         name = backend or cfg.get("backend", "local")
101         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102         settings.update(overrides)
103         return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107               config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
109         (no copy); for s3/gcs it downloads to ``dst``."""
110         ref = parse_uri(uri)
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
112
113
114     def put(local_path: "str | os.PathLike[str]", uri: str, *,
115             config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
116         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
117         ref = parse_uri(uri)
118         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
```

23. assistant (2026-06-21T09:17:46.186Z)

I have everything I need. Key finding on the seam: `acquire_local_input` (in `backend/storage/__init__.py:74`) calls the bare name `fetch` ? which resolves to the `fetch` attribute of the `backend.storage` package module. Since `backend/main.py` does `from backend import storage` and calls `storage.acquire_local_input(...)`, the monkeypatchable seam is `backend.storage.fetch` (i.e. `m.storage.fetch`), exactly as `test_process_nonlocal_uri_skips_exists_and_fetches` already patches it. This is the cleanest hermetic seam for an in-process TestClient guard.

For a subprocess #281-style guard, that monkeypatch doesn't reach a real subprocess ? there you'd need either a real local-mount fake (e.g. a `gdrive://` over a tmp mount, which needs no SDK) or inject a fake backend; but the engine maps `gs://`?`gcs` backend which requires `google-cloud-storage` unless a client is injected, which a subprocess can't easily do. The in-process TestClient path is the hermetic fit for `gs://`.

Now I'll return the structured answer.

24. user (2026-06-21T09:18:21.308Z)

Structured output provided successfully

25. assistant (2026-06-21T09:18:35.640Z)

I've mapped the test surface and returned the structured answer. Summary of the key findings:

1. **Yes, a gs:// end-to-end test already exists** ? `test_process_nonlocal_uri_skips_exists_and_fetches` (`C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_api_endpoints.py:375-398`) drives `gs://bucket/videos/x.mp4` through `/api/process` to a `done` job and asserts the API-DB record keeps the source URI. The gap: it does not additionally assert `db.query_play_segments(video_id, {})` is non-empty for the gs:// case, nor pin both `gs://` and `gcs://` schemes, nor confirm `video_id` is the hash of the fetched bytes.

2. **Best pattern to mirror**: the M2 block in `tests/test_api_endpoints.py` with its `env` fixture (real `Database` + `_InlineExecutor` making the job synchronously `done`). The literal #281 subprocess guard is `tests/test_server_entrypoint.py`, but it can't inject fakes so it can't host a hermetic gs:// test.

3. **Cleanest hermetic seam**: `monkeypatch.setattr(m.storage, "fetch", fake_fetch)` ? because `acquire_local_input` (`backend/storage/__init__.py:74`) calls the bare module-level `fetch`, and `backend/main.py:159` reaches it via `storage.acquire_local_input(...)`. Patching there intercepts before any `get_storage`/`_backend_class('gcs')`/google-cloud-storage import.

4. **Suite invocation**: `python run_all_tests.py` (passes args through to `pytest -q`); no `conftest.py` exists ? the `env` fixture is duplicated inline per file; TestClient tests `importorskip('httpx')`.